

QuickBite: Viva Notes

These notes explain every design decision in plain English, so you can defend the project in your viva. Read the **one-minute pitch** first. Then go through the sections. Finally, test yourself with the **likely questions** at the end. Try answering each one out loud before you open its answer.

The one-minute pitch

"QuickBite is a food-delivery aggregator. The database stores restaurants, their menus, customers, delivery agents, orders and payments. It has **7 tables in BCNF**. The original data repeated each restaurant's cuisine and address on every menu row, so I moved those into a separate `restaurant` table, which is the 2NF fix the brief asks for. Orders and dishes are many-to-many, so an `order_item` table sits between them. Every order **must** reference a valid customer, restaurant and delivery agent: the FKs are NOT NULL with ON DELETE RESTRICT. A trigger guarantees that every dish on an order comes from that order's restaurant.

For the required query, I find each restaurant's most-ordered dish using a JOIN, GROUP BY, COUNT and a correlated ranking subquery. It correctly shows ties. Revenue per restaurant is a GROUP BY SUM over a view that calculates order totals.

For the advanced concept, I created a composite index on `restaurant(cuisine, location)`. On a 200,000-row catalogue it changes a full table scan into an index scan, about 13× faster in my latest run."

1. The tables, and why each one exists

Table	Why it exists	Relationship
restaurant	Named in the brief. It holds cuisine and address once , which is the 2NF fix.	parent of menu items and orders
menu_item	Named in the brief. These are the dishes.	Restaurant 1 : M MenuItem
customer	Named in the brief.	Customer 1 : M FoodOrder
delivery_agent	Named in the brief.	DeliveryAgent 1 : M FoodOrder
food_order	Named in the brief. It contains OrderID, CustomerID, RestaurantID, OrderTime and DeliveryAgentID, exactly as required.	child of three parents
order_item	Added by me. One order has many dishes and one dish is in many orders, which is M : N. A relational database can't store M : N directly, so a junction table splits it into two 1 : M relationships. The brief calls these "order-lines".	FoodOrder M : N MenuItem
payment	Added by me. The brief asks for 1 : 1, 1 : M and M : N relationships. Payment is a natural 1 : 1: each order has exactly one payment.	FoodOrder 1 : 1 Payment

If they ask "why did you add tables that weren't in the brief?": "The brief said to show 1:1, 1:M and M:N. M:N needs a junction table, which also serves as the order-lines the required query JOINS on. Payment

gave a genuine 1:1. Without these two, the relationships couldn't be modelled correctly."

2. Keys and constraints, and why each one was chosen

Choice	Reason
Surrogate integer PKs with <code>GENERATED ... AS IDENTITY</code>	Natural keys like a name or phone number can change. An auto-number never changes and is fast to join on. <code>IDENTITY</code> is the SQL-standard replacement for <code>SERIAL</code> .
Composite PK (<code>order_id</code> , <code>menu_item_id</code>) in <code>order_item</code>	It identifies a line by its order and dish, and stops the same dish appearing twice in one order. To order more of it, you increase <code>quantity</code> .
FK + UNIQUE on <code>payment.order_id</code>	A plain FK allows many payments per order (1 : M). Adding <code>UNIQUE</code> limits it to one, which is how a 1 : 1 is built.
NOT NULL on all three FKs in <code>food_order</code>	The brief says <i>every</i> order ties to a valid restaurant, customer and agent. An FK accepts <code>NULL</code> , so <code>NOT NULL</code> is what makes the tie compulsory.
ON DELETE RESTRICT on parents	You can't delete a restaurant or customer who has order history. Losing history would break the revenue reports. Mark them inactive (<code>is_active</code>) instead.
ON DELETE CASCADE from order → order lines and payment	Lines and the payment are part of the order. They mean nothing without it.

UNIQUE on email and phone	One account per email or phone. These are also candidate keys , which the BCNF check needs to list.
CHECK constraints	They enforce valid values at the database level: price > 0, rating 0–5, quantity 1–50, allowed statuses and methods, and delivery after the order time. The app's code can have bugs, so the database is the last line of defence.
NUMERIC (8, 2) for money	FLOAT stores approximations: 0.1 + 0.2 ≠ 0.3. Money must be exact.
Phone numbers as VARCHAR	They can start with 0, they're never used in arithmetic, and they may contain +.

3. Normalization: the 2NF story

The problem in the brief: cuisine and address were repeated on every menu-item row.

- **Key of the flat table:** (RestaurantID, ItemID).
- **The dependency:** RestaurantID → Cuisine, Address depends on **part** of the key. That's a *partial dependency*, so the table breaks 2NF.
- **The anomalies it causes:**
 - **Update:** if a restaurant moves, you must change N rows, and missing one leaves the data contradicting itself.
 - **Insert:** you can't add a restaurant until it has a dish.
 - **Delete:** deleting the last dish also deletes the restaurant.
- **The fix:** move those attributes into restaurant, and link to it from menu_item with an FK.

3NF: no non-key attribute depends on another non-key attribute. That's why order totals aren't stored: they're calculated.

BCNF: in every table, every determinant is a candidate key. For

example, `customer` has the determinants `customer_id`, email and phone, and all three are keys. The full table is in [normalization.md](#).

4. "Isn't this redundancy?": the three tricky columns

Examiners like to point at these. Here are the answers:

Column	Looks redundant because...	Why it isn't
<code>order_item.unit_price</code>	<code>menu_item.price</code> already exists	It's the price when the order was placed . When the menu price changes, old bills must not change. It's a different fact, about the line rather than the dish. Practice exercise 32 proves this.
<code>food_order.delivery_address</code>	<code>customer.address</code> already exists	It's where this order was delivered. If the customer moves, past orders still went to the old address.

Order total (deliberately **not** a column)

—

It's calculated as
SUM(quantity
×
unit_price)
by the view
v_order_tot
Storing it would
be a transitive
dependency and
could go out of
sync.

5. Why a trigger instead of copying a column

- **The rule:** a dish on an order must come from that order's restaurant. You can't order biryani from a pizza place.
- **Option A:** copy `restaurant_id` into `order_item` and use a composite FK. ✗ But `restaurant_id` depends only on `order_id`, which is *part* of the key. That's a partial dependency, so it would break 2NF.
- **Option B (chosen):** a `BEFORE INSERT OR UPDATE` trigger looks up both restaurants and raises an error if they differ. ✓ It keeps 2NF and enforces the rule.
- **The trade-off:** the trigger runs 2 small PK lookups per inserted line. That's negligible.

6. The required query, explained simply

Q1: the most frequently ordered dish for each restaurant.

1. **JOIN** the order lines (`order_item`) to `menu_item` to get each dish's name and restaurant. Also join `food_order` to read its status, and `restaurant` to get its name.

2. **WHERE** `status <> 'Cancelled'`. Cancelled orders don't count as demand.
3. **GROUP BY** restaurant + dish. Now there's one row per dish.
4. **COUNT(*)** = how many orders contained that dish.
5. **HAVING COUNT(*) = (ranking subquery)**. The subquery finds the **highest** dish count *for the same restaurant*. It refers to the outer query's restaurant, which makes it a **correlated subquery**. Only dishes that equal the top count survive.

Ties: if two dishes share the top count, both equal the max and both are shown, like Dragon Wok's Veg Hakka Noodles and Chilli Paneer. I built that tie into the sample data on purpose, to prove the query handles it.

The alternative (Q1b):

- `RANK() OVER (PARTITION BY restaurant_id ORDER BY count DESC)` gives the same result.
- `ROW_NUMBER()` picks exactly **one** dish per restaurant, for the promotion banner. It breaks ties by units sold.

COUNT(*) vs SUM(quantity): "most frequently ordered" means *in the most orders*, so I count lines. I also show units sold, because one order of 6 vada pavs isn't the same as 6 people each ordering one.

7. Revenue per restaurant

- This is a **GROUP BY restaurant + SUM(order_total)** query, with order totals coming from the view. Only `Delivered` orders count, since cancelled or unfinished orders haven't earned money.
 - **LEFT JOIN** keeps Malvani Tadka, which has no orders, at ₹0. An **INNER JOIN** would hide it. Q2b shows this: it has 10 rows instead of 11.
 - The status filter sits in the **ON** clause, not in **WHERE**. In **WHERE**, it would remove the **NULL** rows and silently turn the **LEFT JOIN** into an **INNER JOIN**.
-
-

8. The index: questions you'll definitely get

What it is: a B-tree index on `restaurant (cuisine, location)`. It's like a phone book sorted by surname, then by first name.

Why it's needed: without it, the search "North Indian in Kharghar" reads every row, which is a **sequential scan**. With it, the database jumps straight to the matching entries.

The proof (05_index_demo.sql):

1. Load 200,000 restaurants inside a transaction.
2. Run `EXPLAIN ANALYZE` without the index: **Seq Scan**, reading about 200k rows.
3. Run it with the index: **Bitmap Index Scan**, reading only the 835 matches. Roughly **10–30× faster**. The exact milliseconds change from run to run; the README shows the latest run.
4. `ROLLBACK` removes the test rows.

Why the column order is (cuisine, location):

- **Leftmost prefix:** the index helps searches on `cuisine` alone, and on `cuisine + location`.
- It can't directly help `location` alone, just as a surname-sorted phone book can't quickly find everyone called "Rahul".
- *Bonus point:* PostgreSQL 18 added **skip scan**, which can still use the index for location-only searches by jumping to each distinct cuisine. It's slower than a direct seek. The demo shows "Index Searches: 25".

When the index is *not* used:

- **Tiny tables.** With 11 rows, one page read is cheaper than using the index, so the planner chooses a Seq Scan. That's correct behaviour.
- A search on `location` alone (except PG 18 skip scan).
- A function wrapped around the column, e.g. `WHERE LOWER(cuisine) = ....`
- A leading wildcard, e.g. `LIKE '%Indian'`.
- When the condition matches most of the table.

What indexes cost: extra disk space, and every INSERT or UPDATE must also update the index. That's why I index the search columns and FKs, not everything.

Why there are extra FK indexes: PostgreSQL doesn't index FK columns automatically, unlike MySQL. Without these indexes, JOINS and RESTRICT checks scan the child table.

9. Proving the data integrity

`06_integrity_tests.sql` tries 9 bad writes, and **all 9 are rejected**:

Bad write	Rejected by
An order for a customer that doesn't exist	FK
An order for a restaurant that doesn't exist	FK
An order with no delivery agent	NOT NULL
A pizza on a biryani order	trigger
Deleting a restaurant that has data	RESTRICT
A second payment for the same order	UNIQUE (1 : 1)
A negative price	CHECK
A made-up order status	CHECK
A duplicate email	UNIQUE

Each test runs inside its own savepoint, so nothing changes in the database.

10. "What would you change at 10× or 100× the scale?"

- **Partition** `food_order` by month, so old months can be archived and queries on recent data stay fast.
 - Turn `v_order_total` into a **materialized view** (refreshed periodically) or a maintained summary table, so revenue reports don't re-sum millions of lines.
 - Add an index on `food_order(order_time)` for date-range reports, and on `(restaurant_id, status)` for revenue.
 - Move `location` and `cuisine` into lookup tables (`area, cuisine`) with FKs, to stop spelling variants like "Vashi" vs "vashi".
 - Add a `restaurant_location` table, or PostGIS, for real distance-based search.
 - Use read replicas for the reporting queries.
-

11. Likely viva questions: try answering first

Q1. What is the difference between a primary key and a unique key?

Both prevent duplicates. A PK is also NOT NULL, and a table has only **one** PK. A table can have many UNIQUE constraints, and they allow NULLs. In this project, `customer_id` is the PK, while `email` and `phone` are UNIQUE: they're candidate keys that weren't chosen as the PK.

Q2. How did you implement the M:N relationship?

With the junction table `order_item(order_id, menu_item_id, quantity, unit_price)`. Both columns are FKs and together they form the composite PK. It turns one M:N into two 1:M relationships. `quantity` and `unit_price` belong to the relationship itself, not to either entity.

Q3. How did you enforce the 1:1 relationship?

`payment.order_id` is a foreign key **and** UNIQUE. The FK ensures the order exists. UNIQUE ensures no order has two payments.

Q4. Which normal form is your schema in? How do you know?

BCNF. For every non-trivial functional dependency in every table, the left side (the determinant) is a candidate key. For example, in `customer` the only determinants are `customer_id`, `email` and `phone`, and all three are candidate keys. The full table is in `normalization.md`.

Q5. Explain 2NF using your project.

2NF means no non-key attribute depends on only *part* of a composite key. In the flat menu sheet, the key was (RestaurantID, ItemID), but Cuisine and Address depended on RestaurantID alone. I moved them into `restaurant`, which removed the partial dependency along with its update, insert and delete anomalies.

Q6. What is a correlated subquery? Where did you use one?

It's a subquery that refers to a column of the outer query, so it's evaluated once per outer row or group. In Q1, the ranking subquery uses `WHERE m2.restaurant_id = r.restaurant_id` to find the top dish count *for that restaurant*. Q5's NOT EXISTS is also correlated.

Q7. WHERE vs HAVING?

WHERE filters individual rows **before** grouping, and can't use aggregates. HAVING filters groups **after** GROUP BY, and can use aggregates. In Q1, `WHERE status <> 'Cancelled'` removes rows, and then `HAVING COUNT(*) = (...)` keeps only the top groups.

Q8. INNER JOIN vs LEFT JOIN? Give an example from your output.

INNER keeps only rows that match on both sides. LEFT keeps every row from the left table and fills in NULLs where there's no match. The revenue report (Q2, LEFT JOIN) shows Malvani Tadka at ₹0. Q2b (INNER JOIN) leaves it out: 10 rows instead of 11.

Q9. Why is the order total not stored in food_order?

It's derived from the order lines. Storing it creates a transitive dependency, and it can go stale if a line changes, which is an update anomaly. The view `v_order_total` calculates it on demand, so it's

always correct.

Q10. Why use a trigger? Couldn't you use a foreign key?

An FK only checks that a dish exists, not that it belongs to the order's restaurant. A composite FK could, but only if `restaurant_id` were copied into `order_item`, which would break 2NF. The trigger enforces the rule without any redundancy.

Q11. What's a view? Does it store data?

A view is a saved SELECT query that you use like a table. A normal view stores **no data**: it runs the query each time it's read. A *materialized* view stores the result and must be refreshed.

Q12. How does your index make search faster? When won't it be used?

The B-tree is sorted by (cuisine, location), so the database seeks directly to the matching entries instead of scanning every row. In my demo with 200k rows, it went from a Seq Scan to a Bitmap Index Scan, roughly 10–30× faster. It isn't used on tiny tables, on a location-only search (except PG 18 skip scan), when a function is wrapped around the column, or with a leading-wildcard LIKE.

Q13. What does EXPLAIN ANALYZE do?

EXPLAIN shows the execution plan the optimizer chose: Seq Scan, Index Scan, join method and so on. Adding ANALYZE actually **runs** the query and reports real row counts and times.

Q14. What do ON DELETE RESTRICT and ON DELETE CASCADE mean?

RESTRICT blocks deleting a parent that still has children: you can't delete a restaurant with dishes or orders. CASCADE deletes the children along with the parent: deleting an order deletes its lines and its payment.

Q15. What is a transaction? Where did you use one?

A transaction is a group of statements that succeed or fail together, which is the A (atomicity) in ACID. The seed file is wrapped in BEGIN...

COMMIT, so the data loads completely or not at all. The index demo uses BEGIN...ROLLBACK, so 200k test rows are added and then removed without a trace.

Q16. How do you handle ties for the most popular dish?

The ranking subquery keeps **every** dish whose count equals the maximum, so ties are shown (Dragon Wok has two). For one promoted dish, Q1b uses ROW_NUMBER() and breaks ties by units sold, then alphabetically.

Q17. RANK vs DENSE_RANK vs ROW_NUMBER?

For values 10, 10, 8:

- ROW_NUMBER gives 1, 2, 3. It's always unique.
- RANK gives 1, 1, 3. Ties share a rank, then a number is skipped.
- DENSE_RANK gives 1, 1, 2. Ties share a rank, and no number is skipped.

Q18. Why NUMERIC and not FLOAT for price?

FLOAT is binary floating point and can't represent many decimals exactly, which causes rounding errors in money sums. NUMERIC is exact decimal arithmetic.

Numbers to remember

Fact	Value
Tables	7
Normal form	BCNF
Restaurants / dishes / customers / agents	11 / 53 / 15 / 8
Orders	49: 45 delivered, 2 cancelled, 2 in progress

Order lines / payments	110 / 49
Top revenue	Biryani Darbar, ₹5,270
Restaurant with no orders	Malvani Tadka (₹0)
The tie	Dragon Wok: Veg Hakka Noodles and Chilli Paneer, 3 orders each
Index demo	200,000 rows, Seq Scan → Bitmap Index Scan, ~10–30× faster
Integrity tests	9 / 9 rejected